INFORMATION SECURITY

Laboratory Work No. 1

Secure REST API Development with CI/CD and Real CVE Vulnerability Analysis

Variant	**0**
Surname and Name	**Wang Ruilin**
HDU number	**22320110**
Institute	**HDU-ITMO Joint Institute**
Course	**Information Security**
Date	**October 19, 2026**

1. Purpose of the Work

The purpose of this work is to go through the full security lifecycle of a small backend service rather than a single isolated exercise: design an API that resists the OWASP Top 10 patterns, put automated security checks into the delivery pipeline, then take a real, publicly documented vulnerability, reproduce it in a controlled environment, explain why it exists, fix it and prove the fix. The same four skills — build, check, break, repair — are what a security engineer does on a daily basis, and the last two are the ones normally skipped by students.

2. Delivered Project

2.1 Stack

Item	Choice	Reason
Language	JavaScript (Node.js 24)	required by the lab; one language for API and tooling
Framework	Express 4.21	mature, explicit middleware model
Database	SQLite via built-in node:sqlite	real SQL engine, zero native build steps
Authentication	JWT (HS256) + bcrypt	stateless tokens, slow salted password hashes
SAST	ESLint 9 + eslint-plugin-security	14 Node-specific security rules
SCA	npm audit	advisory database maintained by npm

Deliverable links. The repository and the pipeline run are produced from this folder; the two addresses are filled in after the first push, because a public remote requires the student's own GitHub account:

```
Repository:  <to be filled after git push>
Pipeline:    <to be filled after the first Actions run>
```

2.2 API surface

Three endpoints are required by the lab; two more were added because they make the ownership model testable.

#	Method and path	Auth	Purpose
1	POST /auth/login	—	authenticate, returns a JWT
2	GET /api/data	Bearer	returns the caller's notes
3	POST /api/notes	Bearer	creates a note (own choice)
+	GET /api/notes/:id	Bearer	reads one note, scoped to its owner
+	GET /health	—	liveness probe

Example interaction, as recorded while testing:

```
$ curl -s -X POST http://127.0.0.1:3000/auth/login \
       -H 'Content-Type: application/json' \
       -d '{"username":"alice","password":"Alice#2026pass"}'
{"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...","token_type":"Bearer",
 "expires_in":3600,"user":{"id":1,"username":"alice","role":"user"}}

$ curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:3000/api/data
401                                  # no token -> rejected

$ curl -s http://127.0.0.1:3000/api/data -H "Authorization: Bearer $TOKEN"
{"notes":[{"id":1,"owner_id":1,"title":"Welcome",...}]}
```

Terminal transcript: login, rejection without a token, and an authorised read.

3. Implemented Security Measures

3.1 SQL injection

Every statement is prepared once with a constant SQL text and bound parameters; request data never becomes part of the SQL string. This is the only correct defence — escaping quotes is not, because numeric contexts, LIKE clauses and identifier positions all break naive escaping.

```
// src/db.js -- the SQL text is a literal, only the parameters vary
findUserByUsername(db, username) {
  return db.prepare('SELECT id, username, password_hash, role FROM users WHERE username = ?')
           .get(username);
}
```

Code: the login lookup is a prepared statement.

3.2 Cross-site scripting

Two independent layers are used. Input validation rejects values that do not look like a username and enforces length limits on note fields. Output encoding then HTML-escapes every user-controlled string on the way out, so that even a stored payload arrives at the client as inert text.

```
// src/security.js
function encodeForOutput(value) {
  if (typeof value === 'string') return escapeHtml(value);
  if (Array.isArray(value)) return value.map(encodeForOutput);
  if (value && typeof value === 'object') { /* recurse over own columns */ }
  return value;
}
```

Code: recursive output encoding applied to every response body.

Measured effect: the note body <script>alert("xss")</script> is stored verbatim but returned as <script>alert("xss")</script>, and the automated test asserts that the raw tag never appears in any response.

3.3 Broken authentication

Passwords are hashed with bcrypt (cost 10) and never stored or returned in plaintext. A successful login issues a JWT signed with HS256, containing only the subject, username and role, with a one-hour expiry and an issuer claim.

```
jwt.sign({ sub: String(user.id), username: user.username, role: user.role },
         secret, { algorithm: 'HS256', expiresIn: '1h', issuer: 'secure-notes-api' });
```

Code: token issuing.

The verification middleware checks the Bearer scheme, verifies the signature with an explicit algorithm allowlist — which defeats both the alg:none attack and algorithm confusion — validates exp and iss, and then reloads the account from the database so that a token belonging to a deleted user stops working at once.

Two further details are worth naming, because they are what separates a working login from a correct one. First, the handler always performs a bcrypt comparison, even for a username that does not exist, against a dummy hash: without this, the response time would reveal which accounts are registered. Second, five failed attempts per username within fifteen minutes produce HTTP 429, which turns an unlimited online guessing oracle into a rate-limited one.

3.4 Summary

OWASP category	Measure	Where
A03 Injection (SQLi)	prepared statements for every query	src/db.js
A03 Injection (XSS)	input validation + recursive output encoding	src/security.js, src/app.js
A07 Auth failures	bcrypt hashes, JWT with exp/iss, algorithm allowlist, user reload	src/app.js
A01 Broken access control	notes always filtered by owner_id (no IDOR)	src/db.js
A05 Security misconfiguration	helmet headers, 10 kB body limit, generic errors	src/app.js
A02 / A06	no secrets in code, JWT_SECRET mandatory in production	src/app.js

4. CI/CD Pipeline

The pipeline is defined in .github/workflows/security-ci.yml and runs on every push and every pull request. It has three gating stages and one reporting stage.

Stage	Tool	Gate
Tests	npm test (14 checks)	build fails on any failed check
SAST	ESLint 9 + eslint-plugin-security	build fails on any error
SCA	npm audit	build fails on high/critical in production dependencies
Reports	JSON artifacts	uploaded for review

Current state of both scanners on this codebase:

```
$ npx eslint src scripts --format stylish
✖ 1 problem (0 errors, 1 warning)

$ npm audit --registry=https://registry.npmjs.org
found 0 vulnerabilities
```

Terminal transcript: SAST and SCA results. Full JSON reports are in evidence/.

The single SAST warning is security/detect-object-injection in the output encoder. It is a false positive — the object keys are our own column names, not request data — and it is suppressed in place with a one-line justification rather than by disabling the rule for the whole project, so the rule keeps protecting the rest of the code.

One practical note that cost time and is worth recording: npm audit needs the registry advisory endpoint, and a mirror such as npmmirror answers [NOT_IMPLEMENTED] for /-/npm/v1/security/advisories/bulk. The audit stage therefore pins registry.npmjs.org explicitly; otherwise the SCA gate silently degrades into a command that always fails, and a check that always fails is soon ignored.

5. CVE Analysis

5.1 Identification

Field	Value
CVE ID	CVE-2017-14849
Affected product	Node.js 8.5.0 (8.5.0 <= version < 8.6.0)
Weakness	CWE-22 Improper Limitation of a Pathname to a Restricted Directory
Impact	arbitrary file read outside the web root
Fixed in	Node.js 8.6.0 (September 2017)
Scenario used	vulhub/node/CVE-2017-14849 — Express 4.15.5 static server

5.2 The defect in one sentence

Node.js 8.5.0 rewrote how path.normalize() collapses .. segments and got it wrong when a .. follows an ordinary segment in the middle of a path, so the function silently removes traversal segments that should have survived; every framework that uses normalize() to keep file access inside a directory — express.static through the send module is the textbook case — then serves files from outside that directory.

5.3 Environment and reproduction

Docker is not installed on the machine used for this work, so the vulhub scenario was reproduced natively: the same application source and the same runtime version, started directly instead of in a container. The vulnerability lives in the Node.js runtime, so the packaging is irrelevant to the result. The exact Docker commands are given for completeness.

```
# official path (needs Docker)
git clone https://github.com/vulhub/vulhub.git
cd vulhub/node/CVE-2017-14849 && docker compose up -d

# path actually executed here
cd cve/CVE-2017-14849 && npm install          # express 4.15.5
curl -L --ssl-no-revoke -o node8.zip \
```

```
  https://registry.npmmirror.com/-/binary/node/v8.5.0/node-v8.5.0-win-x64.zip
tar -xf node8.zip -C C:\n850
C:\n850\node-v8.5.0-win-x64\node.exe www/app.js
# Example app listening on port 3000!
```

Terminal transcript: bringing up the vulnerable target.

Baseline and attack. The static asset is served normally; the traversal request reads a file that sits two directories above the served root:

```
$ curl --path-as-is -s -o /dev/null -w '%{http_code}\n' \
       http://127.0.0.1:3000/static/main.js
200

$ curl --path-as-is -s -i 'http://127.0.0.1:3000/static/../foo/../../secret.txt'
HTTP/1.1 200 OK
X-Powered-By: Express
Accept-Ranges: bytes
Content-Type: text/plain; charset=UTF-8
Content-Length: 57

SECRET_FLAG{this-file-lives-outside-the-static-root}
```

Terminal transcript: successful path traversal. secret.txt lives outside www/static.

Two reproduction details matter. curl normalises .. in a URL by default, so without --path-as-is the payload never leaves the client and the attack appears to fail. And on Windows the payload shape differs from the Linux PoC in the vulhub README, because the absolute path is built from a drive-letter root; /static/../foo/../../secret.txt is the form that escapes here.

5.4 Root cause

The guard inside send@0.15.6 is a normalise-then-check pattern:

```
// node_modules/send/index.js
if (UP_PATH_REGEXP.test(normalize('.' + sep + path))) return this.error(403);  // L538
path = normalize(join(root, path));                                             // L545
```

Code: the check is only as sound as normalize().

The pattern assumes that a surviving .. segment is the signature of an escape attempt. Node 8.5.0 breaks that assumption:

```
$ node-v8.5.0 -e "console.log(require('path').normalize('../foo/../../secret.txt'))"
secret.txt

$ node       -e "console.log(require('path').normalize('../foo/../../secret.txt'))"
..\..\secret.txt
```

Terminal transcript: the runtime disagrees with itself. Two leading .. segments vanish.

After normalisation there is nothing left for the regular expression to match, so the request is classified as a clean relative path and join(root, 'secret.txt') is resolved one level above root.

Classifying the error, which is what the lab asks for: this is a logic error in a language runtime library. It is not a missing input filter and not a misconfiguration. The application used the standard library exactly as documented; the library returned the wrong answer. That classification is what tells us where the fix has to go.

5.5 Fix

Because the defect is in the runtime, the fix is a runtime upgrade and the application source does not change at all. The files in fixes/ contain the changed artefacts.

```
# fixes/Dockerfile.node-8.6.0   -- minimal fix
FROM node:8.6.0

# fixes/Dockerfile.node-20      -- recommended fix: supported LTS, non-root user
FROM node:20-alpine
USER node
```

Code: the entire patch is the base image tag.

This is effective rather than a workaround for a specific reason. The send guard rests on an invariant of the standard library; restoring the invariant repairs every library that depends on it, not just this application. The alternative — filtering traversal sequences at the request boundary — would mean re-implementing path semantics in application code, which is precisely how the next traversal bug is born.

5.6 Verification and regression

The same application was restarted on a fixed runtime and the identical request replayed:

```
$ node www/app.js                       # Node 24
$ curl --path-as-is -s -i 'http://127.0.0.1:3000/static/../foo/../../secret.txt'
HTTP/1.1 404 Not Found

# regression: legitimate functionality still works
$ curl --path-as-is -s -o /dev/null -w '%{http_code}\n' \
       http://127.0.0.1:3000/static/main.js
200
$ curl --path-as-is -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:3000/
200
```

Terminal transcript: attack blocked, service intact.

A useful cross-check comes from SCA. Running the same dependency audit against the pinned vulnerable tree reports eight advisories, two of them critical — the signature of an end-of-life runtime — while the API project reports zero. The upgrade therefore fixes this CVE and removes the wider exposure at the same time.

```
$ cd cve/CVE-2017-14849 && npm audit --registry=https://registry.npmjs.org
8 vulnerabilities (2 low, 4 high, 2 critical)

$ cd .. && cd .. && npm audit --registry=https://registry.npmjs.org
found 0 vulnerabilities
```

Terminal transcript: SCA comparison between the vulnerable tree and the API project.

6. Control Questions

1. Why is bcrypt preferable to SHA-256 for password storage?

SHA-256 is designed to be fast, so an attacker with a stolen database can test billions of candidates per second on commodity GPUs. bcrypt is deliberately slow, tunable through its cost factor, and salts every hash automatically, so the same password produces different digests and precomputed rainbow tables are useless. A plain hash also has no work factor at all, so it cannot be strengthened as hardware gets faster.

2. What is the main difference between SAST and DAST?

SAST analyses source code without running it, so it sees every branch including code that never executes, but it cannot tell whether a finding is reachable from an entry point and therefore produces false positives. DAST attacks a running application from the outside and reports only vulnerabilities that are actually exploitable, but it can only see what it manages to reach and gives no line number. This work uses SAST in the pipeline and relies on the CVE reproduction for the dynamic side.

3. Describe how a JWT works, what it contains, and how the server verifies it.

A JWT is three base64url parts: a header naming the algorithm, a payload of claims, and a signature over the first two. Our payload carries sub, username, role, iat, exp and iss. The payload is only encoded, not encrypted, so nothing secret may be placed in it. The server verifies by recomputing the HMAC with its secret key, restricting the accepted algorithms to HS256, checking exp and iss, and then reloading the account so that a deleted user's token stops working.

4. What risks arise if third-party libraries are not audited?

Most of the code in a modern service is somebody else's. An unaudited dependency can carry a publicly known vulnerability into production — exactly what happened in the CVE part of this work, where an outdated express tree reported eight advisories including two critical — or it can be taken over through a compromised maintainer account and start exfiltrating data on install. SCA gives early warning; lockfiles and a pinned, reviewed dependency set limit the blast radius.

5. What advantages do Docker and Vulhub give over installing vulnerable software directly?

Precision: Vulhub pins the exact vulnerable version, so the result is reproducible. Isolation: the vulnerable service cannot damage the host or the network, and stopping it is one command. Hygiene: no old runtime is left installed with its own CVEs. Reset: the environment is rebuilt in seconds after being broken. On this machine Docker was unavailable, and the native reproduction needed extra care with path semantics and a temporary interpreter directory — a practical illustration of the same point.

6. Distinguish a vulnerability from an exploit. What was reproduced in this work?

A vulnerability is the flaw itself: here, the incorrect path normalisation in Node.js 8.5.0. An exploit is a concrete input that abuses it: here, the HTTP request GET /static/../foo/../../secret.txt. This work reproduced the exploit in order to demonstrate the vulnerability, but the object of study and the object of repair is the flaw, which is why patching the runtime — not the request — was the fix.

7. Why is regression testing essential after applying a patch?

A fix that breaks the service is indistinguishable from a denial of service. Blocking traversal is trivial if the static server is simply turned off, so proving that /static/main.js and / still return 200 is part of the evidence, not an optional extra. Regression tests are also how a later refactoring is prevented from reintroducing the same behaviour.

8. How does the ability to reproduce CVE vulnerabilities help a security specialist?

It converts a text description into an observable behaviour, which is what is needed to decide whether a given system is really affected, to write a detection rule that fires, to confirm that a patch works, and to explain the risk to people who do not read advisories. In a SOC the same skill is the difference between closing a ticket as 'theoretically vulnerable' and demonstrating exploitation in the logs.

7. Conclusion

Both halves of the work are complete and verified. The API implements the three required endpoints, protects the injectable surfaces with prepared statements and output encoding, and implements authentication with bcrypt hashes and JWTs whose verification path defends against the classic algorithm attacks. Fourteen automated checks cover the properties claimed, and the pipeline enforces SAST and SCA on every push, currently reporting zero ESLint errors and zero npm audit findings.

The CVE part was more instructive than the API part. Reproducing CVE-2017-14849 showed that a vulnerability can live below the code a developer writes: the application was correct, the framework followed its own documentation, and the break was in the meaning of a standard library function. Once that was understood, the fix became obvious and minimal — one base image tag — while the naive alternative, filtering traversal strings in the application, would have traded one invisible bug for a visible one.

The limiting factors of this run are recorded honestly in the README: Docker and git were not available, so the vulnerable environment was reproduced natively with the official Node.js 8.5.0 build, and the repository history has to be created before submission.